# 仿真 2：级联信道的信息传输特性

## 1. 实验目的

如下图所示，模拟黑白格图像在级联信道中的传输效果。统计级联信道输入集合与各个子信道输出集合之间的平均互信息量，对比各个子信道输出图像的质量，从“数”“像”两个角度分析级联信道的信息传输特性，验证信息不增定理。

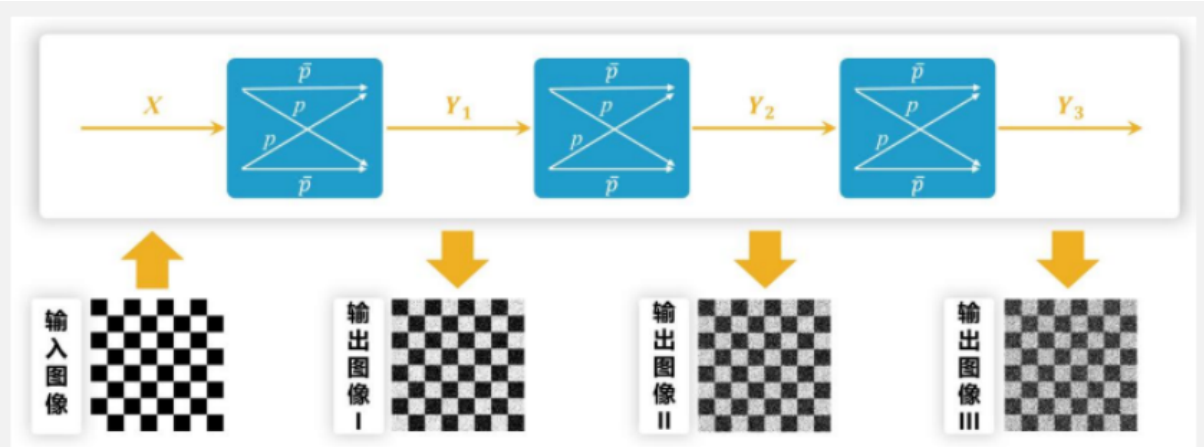



图 1: 黑白图像在实际信道中的传递效果

## 2. 实验原理

利用 Python 编程实现级联信道的仿真，主要包括以下几个步骤：

1. 建立理想信源的输出概率模型 $P(X)$，以及每个级联信道的信道矩阵模型 P

$$P(X) = \begin{bmatrix} \overline{p_i} \\ p_i \end{bmatrix}, \quad \mathrm{P} = \begin{bmatrix} \overline{p}\ ,\ p \\ p\ ,\ \overline{p} \end{bmatrix} \tag{1}$$

2. 利用矩阵乘法，模仿信道的传输过程，计算每个级联信道的输出概率模型 $P(Y)$

$$P(Y) = \mathrm{P}^T \cdot P(X) = \begin{bmatrix} \overline{p}\ ,\ p \\ p\ ,\ \overline{p} \end{bmatrix} \cdot \begin{bmatrix} \overline{p_i} \\ p_i \end{bmatrix} = \begin{bmatrix} \overline{p_i} \cdot \overline{p} + p_i \cdot p \\ p_i \cdot \overline{p} + \overline{p_i} \cdot p \end{bmatrix} = \begin{bmatrix} \overline{p_o} \\ p_o \end{bmatrix} \tag{2}$$

3. 利用互信息量的计算公式，计算每一级输出与原始输入的互信息量

$I(X;Y)$

$$I(X;Y) = H(X) + H(Y) - H(XY) \tag{3}$$

4. 对于图像通信过程模拟，将黑白图像进行二值化处理，展平得到输入序列 $\{x_i\}$，对于每个输入像素点 $x_i$，根据 $x_i$ 的取值按概率对应输出像素点 $x_o$，将输出序列 $\{x_o\}$ 整形为二维图像大小并保存。

## 3. 实验过程

### 3.1 构建信道矩阵模型

根据实验原理，构建二元对称信道的信道矩阵模型 P，并设置可调参数 p_not：

```
def matrix(p_not=0.5):
    return np.array([[p_not, 1-p_not], [1-p_not, p_not]]).flatten()
```

图 2: 二元对称信道的信道矩阵模型

### 3.2 计算互信息量

设置离散信源为等概分布，即：

$$P(X) = \begin{bmatrix} \overline{p_i} \\ p_i \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \tag{4}$$

根据二元对称信道的特性，输出概率分布 $P(Y)$ 为：

$$P(Y) = \mathrm{P}^T \cdot P(X) = \begin{bmatrix} \overline{p}\ ,\ p \\ p\ ,\ \overline{p} \end{bmatrix} \cdot \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.5 \cdot \overline{p} + 0.5 \cdot p \\ 0.5 \cdot \overline{p} + 0.5 \cdot p \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} \tag{5}$$

因此，对于输入等概分布的二元对称信道，输出概率分布也是等概分布，因此：

$$H(X) \equiv H(Y_i) \equiv H(\frac{1}{2}, \frac{1}{2}) = 1 \ \ (i = 1, 2, 3) \tag{6}$$

计算任意级输出 $Y_i$ 与输入的联合概率分布 $P(XY_i)$，进而求得联合熵：

$$P(XY_i) = \mathrm{P}^{(i)} \odot [P(X), P(X)] = \mathrm{P}^{(i)} \odot \begin{bmatrix} 0.5, 0.5 \\ 0.5, 0.5 \end{bmatrix} \tag{7}$$

利用互信息量的计算公式，逐步改变 p_not 的值，计算每个级联信道的互信息量 $I(X;Y_i)$，绘图如下所示：

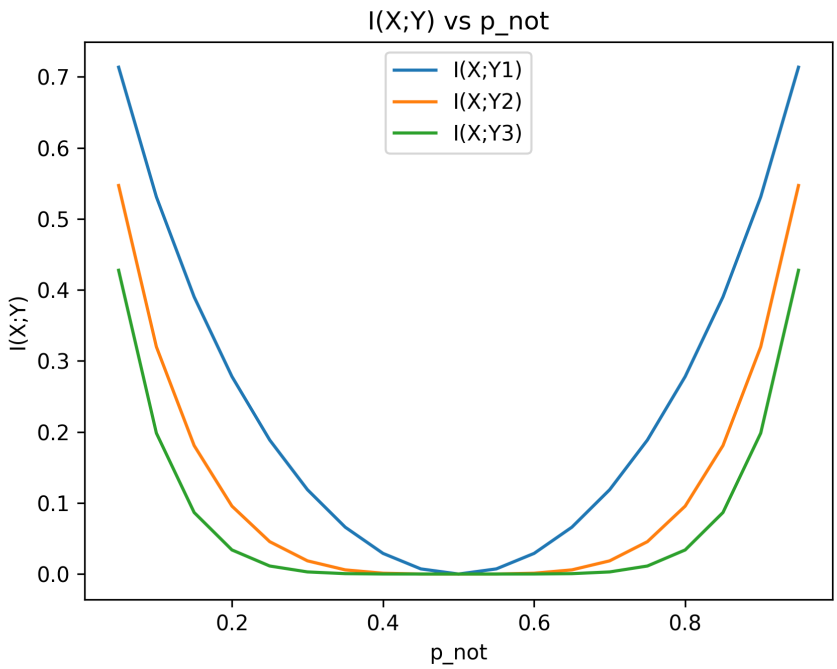



图 3: 互信息量随 p_not 的变化曲线

对于任何一个 p_not 的值，各级输出的互信息量均满足：

$$I(X;Y_1) \geq I(X;Y_2) \geq I(X;Y_3) \tag{8}$$

这是因为，根据信息不增定理，级联信道的互信息量是单调不递增的，因此出现上述情况。

## 3.3 图像传输效果对比

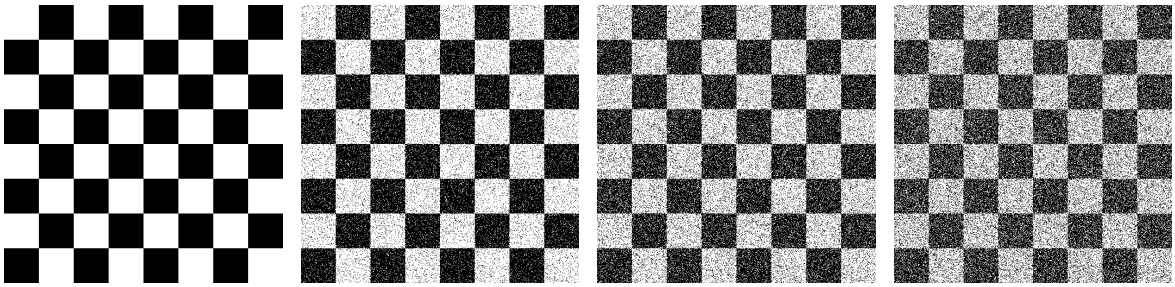
当 p_not=0.9 时，建立等概的输入图像模型 $P(X)$，利用上一问建立的平均互信息量与等概分布信源的计算模型，计算出等概分布的图像信号通过各级信道后的输出图像与输入图像的平均互信息量，结果如下所示。

```
(pythontest) PS D:\zsm_codes\
I(X;Y1) = 0.5310
I(X;Y2) = 0.3199
I(X;Y3) = 0.1984
```

图 4: p_not=0.9 时，各级信道输出图像的与输入图像的理论平均互信息量

### 3.4 图像传输效果

根据实验原理，将读取输入图像并通过各级信道，计算模拟输出图像，结果如下所示：



(a) 输入图像　(b) 一级信道输出　(c) 二级信道输出　(d) 三级信道输出

从实验结果来看，原始输入图像每通过一级信道，都会引入一部分噪声，导致信息损失，输出图像的质量逐级下降。通过对比各级信道输出图像与输入图像的平均互信息量，可以看出，随着信道的级联，互信息量逐渐减小，符合信息不增定理。

## 4. 总结

在本次实验中，我们通过对级联信道的仿真，验证了信息不增定理。通过计算互信息量和图像传输效果，我们观察到随着信道的级联，互信息量逐渐减小，输出图像的质量也逐渐下降。这些结果表明，在实际通信中，信道的噪声会对信息传输产生影响，因此在设计通信系统时，需要考虑信道的特性，以提高信息传输的可靠性。同时，我们也认识到互信息量在信息论中的重要性，它不仅可以用来衡量信源和信道之间的关系，还可以用于评估通信系统的性能。通过本次实验，我们对级联信道的信息传输特性有了更深入的理解，为今后的学习和研究打下了基础。

## 5. 代码附录

Listing 1: 实验代码

```
import numpy as np
from PIL import Image
from utils import p_Markov_topk, H_x
from matplotlib import pyplot as plt
import os

filename = __file__.split('\\')[-1].split('.')[0]

def p_xy(i , k=1 , p_not=0.5):
    k_trans_matrix = np.linalg.matrix_power(matrix(p_not).reshape(2,2), k)
    p_xy = p_Markov_topk(2, 1, i, k_trans_matrix.flatten(), 2)
    return p_xy

def H_xy(i , k=1 , p_not=0.5):
    h_xy = H_x(p_xy(i, k, p_not))
    return h_xy

def matrix(p_not=0.5):
    return np.array([[p_not, 1-p_not], [1-p_not, p_not]]).flatten()

def channel_pass_theory(i, p_matrix):
    p_matrix = p_matrix.reshape(-1, i.shape[0]).T
    return p_matrix @ i.reshape(-1, 1)

def channel_pass(i, p_matrix):
    p_matrix = p_matrix.reshape(2,2)
    return np.array(list(map(lambda x:
                            np.random.choice([0, 1], p=p_matrix[x]), i)))

def png2npy(image_path):
    image = Image.open(image_path).convert('L')
    array = np.array(image, dtype=np.uint8).flatten() // 255
    return array

def npy2png(array, image_path, h=512, w=512):
    if array.shape[0] != h * w:
        raise ValueError("Array size does not match the height and width.")
    array = array.astype(np.uint8) * 255  # 将0和1转换为0和255
    image = Image.fromarray(array.reshape(h, w))
    image.save(image_path)


def question2():
```

```
    i = np.array([0.5, 0.5])
    h_x = H_x(i)
    plt.cla()
    y1, y2, y3 = [],[],[]
    p_not_array = np.arange(0.05, 1, 0.05)
    for p_not in p_not_array:
        o1 = channel_pass_theory(i, matrix(p_not))
        o2 = channel_pass_theory(o1, matrix(p_not))
        o3 = channel_pass_theory(o2, matrix(p_not))
        p_xy1 = p_Markov_topk(2, 1, i, matrix(p_not).flatten(), 2)
        p_xy2 = p_Markov_topk(3, 1, i, matrix(p_not).flatten(), 2)\
                                .reshape(2,-1, 2).sum(axis=1).flatten()
        p_xy3 = p_Markov_topk(4, 1, i, matrix(p_not).flatten(), 2)\
                                .reshape(2,-1, 2).sum(axis=1).flatten()
        I1 = H_x(o1) + h_x - H_x(p_xy1)
        I2 = H_x(o2) + h_x - H_x(p_xy2)
        I3 = H_x(o3) + h_x - H_x(p_xy3)
        y1.append(I1)
        y2.append(I2)
        y3.append(I3)
    plt.plot(p_not_array, y1, label='I(X;Y1)')
    plt.plot(p_not_array, y2, label='I(X;Y2)')
    plt.plot(p_not_array, y3, label='I(X;Y3)')
    plt.xlabel('p_not')
    plt.ylabel('I(X;Y)')
    plt.title('I(X;Y)␣vs␣p_not')
    plt.legend()
    plt.savefig('./实验二/I(X;Y).png', dpi=300)


def question3_4():
    p_not = 0.9
    p_i = np.array([0.5, 0.5])
    h_x = H_x(p_i)
    p_o1 = channel_pass_theory(p_i, matrix(p_not))
    p_o2 = channel_pass_theory(p_o1, matrix(p_not))
    p_o3 = channel_pass_theory(p_o2, matrix(p_not))
    p_xy1 = p_Markov_topk(2, 1, p_i, matrix(p_not).flatten(), 2)
    p_xy2 = p_Markov_topk(3, 1, p_i, matrix(p_not).flatten(), 2)\
                                .reshape(2,-1, 2).sum(axis=1).flatten()
    p_xy3 = p_Markov_topk(4, 1, p_i, matrix(p_not).flatten(), 2)\
                                .reshape(2,-1, 2).sum(axis=1).flatten()
    I1 = H_x(p_o1) + h_x - H_x(p_xy1)
    I2 = H_x(p_o2) + h_x - H_x(p_xy2)
```

```
    I3 = H_x(p_o3) + h_x - H_x(p_xy3)
    print(f"I(X;Y1)␣=␣{I1:.4f}")
    print(f"I(X;Y2)␣=␣{I2:.4f}")
    print(f"I(X;Y3)␣=␣{I3:.4f}")

    i = png2npy('./实验二/黑白格.png')
    o1 = channel_pass(i, matrix(p_not=0.9))
    npy2png(o1, './实验二/o1.png')
    o2 = channel_pass(o1, matrix(p_not=0.9))
    npy2png(o2, './实验二/o2.png')
    o3 = channel_pass(o2, matrix(p_not=0.9))
    npy2png(o3, './实验二/o3.png')


if __name__ == "__main__":
    if not os.path.exists(filename):
        os.mkdir(filename)
    question2()
    question3_4()
```

Listing 2: utils.py 文件

```
import numpy as np
import warnings
# 清楚 divide by zero 的警告
warnings.filterwarnings("ignore")


def p_Markov_topk(k, n, p_n, P_base, x):
    """
        递归计算 k 维 m 阶马尔可夫信源的联合概率分布

    Parameters
    ----------
        k : int
            待计算的维数
        n : int
            已知概率分布的维数    n >= m
        p_n : array_like
            已知概率分布 shape = ( x ** n, )
        P_base : array_like
            转移概率矩阵 shape = ( x ** m, x).fallen() = (x ** (m+1), )
        x : int
            信源符号个数

```

```
    Returns
    -------
        p_k : k 维 m 阶马尔可夫信源的联合概率分布 shape = (x ** k, )
    """
    if k < 1:
        raise ValueError("k must be postive")
    if k == n:
        p_k = p_n
    elif k < n:
        p = p_Markov_topk(k + 1, n, p_n, P_base, x).reshape(-1, x)
        p_k = np.sum(p, axis=1).reshape(-1)
    else:
        p_topk_1_rsp = p_Markov_topk(k - 1, n, p_n, P_base, x)
        p_topk_1_rsp = p_topk_1_rsp.reshape(-1, x**n)
        sub_list = []
        for i in range(x):
            ans = np.multiply(P_base.reshape(x**n, x)[:, i], p_topk_1_rsp)\
                        .reshape(-1, 1)
            sub_list.append(ans)
        p_k = np.concatenate(sub_list, axis=1).reshape(-1)
    return p_k

def H_x(p):
    """
    计算熵
    Parameters
    ----------
        p : array_like
            概率分布 shape = (x, )
    Returns
    -------
        entropy : float
            熵值
    """
    p = p.flatten()
    return -np.nansum(p * np.log2(p))
```